LinkedIn Job Scout

סוכן חיפוש עבודה אוטומטי מבוסס בינה מלאכותית

מסמך ארכיטקטורה מפורט המתאר את מבנה המוצר, רכיביו, וכיצד הם עובדים יחד כדי למצוא, לדרג ולשלוח עדכוני משרות באופן אוטומטי — בלי שתצטרך לחפש בעצמך.

מרץ 2026

תוכן עניינים

1. מה זה LinkedIn Job Scout? 3
המטרה והבעיה שהוא פותר
איך זה עובד ברמה הגבוהה
2. ארכיטקטורה ומבנה הקבצים 4
מבנה תיקיות הפרויקט
תרשים זרימה — מה קורה בכל ריצה
3. רכיב 1: main.py — המוח המרכזי 5
4. רכיב 2: auth.py — מודול ההתחברות 6
5. רכיב 3: searcher.py — מנוע החיפוש 7
6. רכיב 4: filter.py — מנוע הדירוג (AI) 9
7. רכיב 5: notifier.py — שולח המיילים 11
8. רכיב 6: tracker.py — מסד הנתונים 12
9. רכיב 7: app.py — ממשק הניהול 13
10. config.yaml ו-.env — קבצי ההגדרות 15
11. הגנה מפני זיהוי כבוט 16
12. מדריך התקנה והפעלה 17

פרק 1

מה זה LinkedIn Job Scout?

הבעיה שהמוצר פותר

חיפוש עבודה ב-LinkedIn הוא תהליך מתסכל: צריך להיכנס מדי יום, לחפש, לסנן, ולקרוא דסות פרסומים — רובם לא מתאימים. LinkedIn Job Scout אוטומט את כל הפרק הזה.

הסוכן מתחבר ל-LinkedIn בשמך, מחפש משרות לפי הקריטריונים שלך, שולח כל משרה לבינה מלאכותית שמדרגת אותה מ-0 עד 100 ביחס ל-CV שלך, ואז שולח לך מייל עם רק המשרות הכי רלוונטיות — אתה רק בוחר לאיזה מהן להגיש מועמדות.

✅ **מה הסוכן עושה בשבילך:**

מחפש משרות חדשות → קורא את תיאור כל משרה → מדרג כל משרה עם Claude AI → שולח מייל עם המשרות הטובות ביותר → לא שולח את אותה משרה פעמיים

⚠️ **מה הסוכן לא עושה:**

הסוכן לא מגיש מועמדות בשמך. הוא רק מוצא ומסנן משרות. ההחלטה אם להגיש ואיך להגיש — נשארת אצלך.

איך זה עובד ברמה הגבוהה

פעם אחת בשעה (או לפי הגדרה), הסוכן מבצע את הצעדים הבאים:

1. **מתחבר ל-LinkedIn** — נפתח דפדפן (Chromium) ומתחבר עם הפרטים שלך. אם כבר מחובר מריצה קודמת, משתמש בעוגיות השמורות.
2. **מחפש משרות** — מבצע חיפוש לפי מילות המפתח שהגדרת (לדוגמה: "Software Engineer", "Python Developer") ומסנן לפי מיקום ורמת ניסיון.
3. **אוסף פרטים** — לכל משרה חדשה שלא ראה בעבר, נכנס לדף המשרה וקורא את התיאור המלא.
4. **מדרג עם AI** — שולח כל משרה + ה-CV שלך לבינה המלאכותית Claude, שמחזירה ציון 0-100 והסבר בעברית.

5 **שולח מייל** — רק משרות עם ציון גבוה מהסף שהגדרת נשלחות אליך במייל מעוצב. כל משרה נשלחת רק פעם אחת.

הטכנולוגיות בשימוש:

Python | Playwright | Claude AI | SQLite | Flask | Gmail SMTP

פרק 2

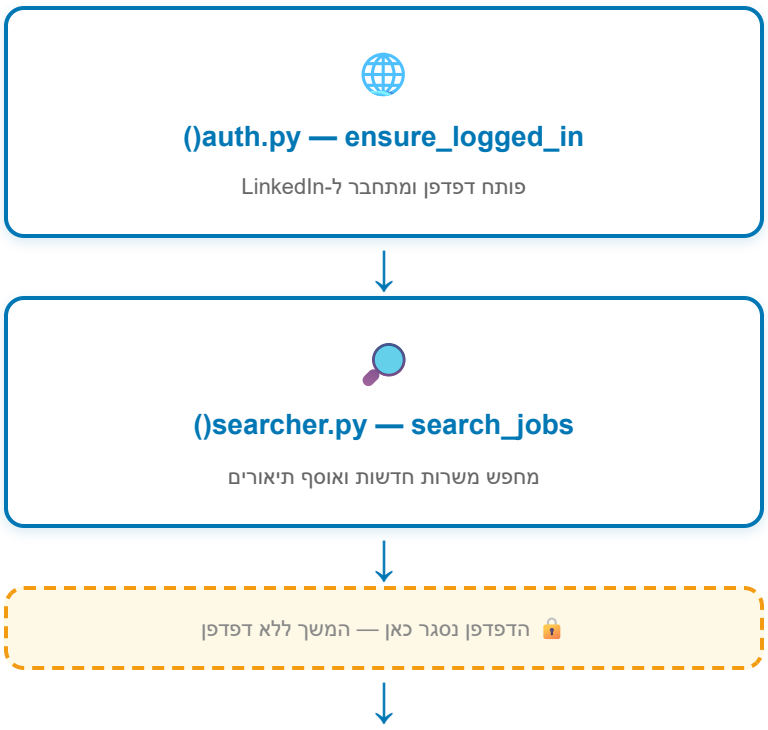
ארכיטקטורה ומבנה הקבצים

מבנה תיקיות הפרויקט

```
LinkedIn/ # תיקיית הפרויקט הראשית
│
├── main.py # נקודת הכניסה — מריץ הכל
├── app.py # ממשק ניהול ווב (Flask)
├── config.yaml # הגדרות: מילות חיפוש, ציון מינימלי...
├── .env # סיסמאות ומפתחות API (פרטי!)
├── cv.txt # ה-CV שלך (טקסט פשוט)
├── requirements.txt # רשימת ספריות Python
├── src/ # מודולים עיקריים
│   ├── auth.py # התחברות ל-LinkedIn
│   ├── searcher.py # חיפוש וסריקת משרות
│   ├── filter.py # דירוג עם Claude AI
│   ├── notifier.py # שליחת מיילים
│   └── tracker.py # מסד נתונים SQLite
├── data/ # נתונים שנוצרים בזמן ריצה
│   ├── jobs.db # מסד נתונים של כל המשרות
│   └── session.json # עוגיות LinkedIn שמורות
└── logs/
    agent.log # לוג של כל הריצות
```

תרשים זרימה — מה קורה בכל ריצה

הפייפליין הוא רצף קבוע שמריץ כל רכיב בזה אחר זה:



()tracker.py — mark_seen
שומר את כל המשרות שנמצאו ב-DB
()filter.py — filter_jobs
שולח לClaude AI לדירוג — מסנן מה מתחת לסף
()notifier.py — send_digest
בונה ושולח מייל HTML עם המשרות הרלוונטיות

פרק 3

main.py — המוח המרכזי

main.py — האורכסטרטור

main.py (192 שורות)

זהו הקובץ הראשי — "המנהל" שמפעיל את כל שאר הרכיבים בסדר הנכון. הוא לא עושה שום דבר בעצמו — הוא רק מתאם בין כל המודולים.

שלושה מצבי הפעלה

מצב Scheduler (ברירת מחדל)

`python main.py`

מריץ את הסוכן מיד ואז שוב כל N שעות (לפי הגדרה). רץ ברקע עד שמפסיקים ידנית.

▶ ריצה חד-פעמית

`python main.py --once`

מריץ פעם אחת, שולח מייל, ומסיים. שימושי לבדיקה ידנית.

Dry Run — ריצת בדיקה ללא מייל

`python main.py --dry-run`

עושה הכל — חיפוש, דירוג — אך לא שולח מייל. מדפיס את התוצאות לטרמינל. מצוין לבדיקת הגדרות לפני שמתחילים "אמיתי".

הפונקציה run_agent() — לוגיקת הריצה

זוהי הפונקציה המרכזית שמנהלת כל ריצה. הנה מה שהיא עושה שלב אחר שלב:

#	שלב	מה קורה	מי עושה
1	אתחול DB	מוודא שמסד הנתונים קיים ותקין	tracker.init_db()
2	טעינת משרות ידועות	טוען את כל המזהים של משרות שכבר נראו	tracker.get_all_seen_ids()
3	פתיחת דפדפן	מפעיל Chromium גלוי (לא headless)	Playwright

#	שלב	מה קורה	מי עושה
4	התחברות	מחובר ל-LinkedIn (עם עוגיות שמורות)	auth.ensure_logged_in()
5	חיפוש	מחפש משרות חדשות לפי כל מילת מפתח	searcher.search_jobs()
6	סגירת דפדפן	הדפדפן נסגר — שאר הפעולות ללא דפדפן	browser.close()
7	שמירת משרות	כל משרה שנמצאה נשמרת כ-"נראתה" בDB	tracker.mark_seen()
8	דירוג AI	Claude מדרג כל משרה — מסנן לפי ציון	filter.filter_jobs()
9	שליחת מייל	שולח מייל עם המשרות שעברו את הסף	notifier.send_digest()
10	עדכון DB	מסמן את המשרות שנשלחו כ-"נשלח"	tracker.mark_emailed()

ה-Scheduler האוטומטי

כשמריצים ללא דגלים, הסוכן משתמש בספריית **APScheduler** כדי לרוץ אוטומטית כל N שעות (לפי `schedule_hours` ב-config.yaml). הריצה הראשונה מתחילה מיידית, ואז ממשיכה לפי לוח הזמנים.

> 📋 **לוגינג — כל ריצה מתועדת**
>
> כל פעולה מוקלטת בלוג `logs/agent.log` ובנוסף מוצגת בטרמינל. הלוג הוא append-only — לא נמחק בין ריצות.

פרק 4

auth.py — מודול ההתחברות

auth.py — ניהול סשן LinkedIn

src/auth.py

מטפל בכל הנוגע לכניסה לחשבון LinkedIn: שמירת עוגיות, טעינת סשן קיים, ביצוע התחברות כשצריך, וזיהוי מצבי 2FA.

לוגיקת ה-"Smart Login"

המודול חכם מספיק לדעת שאין צורך להתחבר בכל פעם. הנה הלוגיקה שהוא עוקב אחריה:

1. מנסה לטעון עוגיות שמורות מ-**data/session.json**
2. נכנס לדף ה-Feed של LinkedIn ובודק אם המשתמש מחובר

אם מחובר: ממשיך ישר לחיפוש — חוסך זמן וחשד

אם לא מחובר: ממלא אוטומטית את שדות המייל וסיסמה ולוחץ כניסה

אם יש 2FA/CAPTCHA: מחכה 60 שניות לסיום ידני של האימות

עוגיות — מה הן ולמה חשובות

"עוגיות" (Cookies) הן קבצוני מידע קטנים שהדפדפן שומר אחרי התחברות מוצלחת. LinkedIn משתמשת בהן כדי לזהות שאתה מחובר — בלי שתצטרך להתחבר שוב.

הסוכן שומר את העוגיות בקובץ JSON אחרי כל התחברות מוצלחת. בריצה הבאה הוא "מעמיס" אותן לדפדפן — כאילו נפתח הדפדפן עם הסשן הקיים. זה חוסך זמן ומקטין את הסיכוי שLinkedIn "יתפוס" שזה בוט.

פונקציות עיקריות

פונקציה	מה עושה
`ensure_logged_in()`	פונקציה ראשית — מריצה את כל הלוגיקה הנ"ל
`save_cookies()`	שומרת עוגיות לקובץ data/session.json
`load_cookies()`	טוענת עוגיות מהקובץ לדפדפן
`is_logged_in()`	בודקת אם הדף הנוכחי מראה משתמש מחובר
`login()`	מבצעת התחברות מלאה עם שם משתמש וסיסמה
`random_delay()`	ממתינה זמן אקראי בין פעולות — נראה אנושי יותר

⚠️ **אבטחה חשובה:**

פרטי ההתחברות (מייל + סיסמה) לעולם לא מאוחסנים בקוד — הם נקראים רק מקובץ ה-.env שנמצא אצלך בלבד.

פרק 5

searcher.py — מנוע החיפוש

searcher.py — Web Scraper

src/searcher.py

המודול שאחראי על חיפוש משרות ב-LinkedIn וקריאת תיאוריהן. משתמש ב-**Playwright** — ספרייה שמפעילה דפדפן אמיתי כדי לגלוש ב-LinkedIn, ממש כמו שאתה היית עושה — רק אוטומטי.

מה זה Playwright ולמה משתמשים בו?

Playwright היא ספרייה שמאפשרת לשלוט בדפדפן (Chrome, Firefox) מקוד Python. אפשר לנווט לכתובות, ללחוץ כפתורים, למלא טפסים ולקרוא תוכן עמודים — הכל אוטומטי. LinkedIn לא מאפשרת גישה ישירה לנתונים שלה דרך API, לכן משתמשים בשיטה זו לקרוא את התוכן מהדף עצמו.

מבנה הנתונים — Job Dataclass

כל משרה שנמצאת מיוצגת כאובייקט Python עם השדות הבאים:

```
@dataclass class Job: job_id: str # מזהה ייחודי של LinkedIn (מספרי) title: str # שם התפקיד company: str # שם החברה location: str # מיקום description: str # תיאור המשרה (מלא) apply_url: str # קישור להגשת מועמדות is_easy_apply: bool # האם יש כפתור "Easy Apply" של LinkedIn
```

תהליך החיפוש — שלב אחר שלב

שלב א׳ — בניית כתובת החיפוש

עבור כל מילת מפתח ב-config, הסוכן בונה כתובת URL של LinkedIn עם פרמטרים של מיקום, רמת ניסיון, ומשרות שפורסמו בשעה האחרונה:

```
# דוגמה לכתובת שנבנית: https://www.linkedin.com/jobs/search/ ?keywords=Software+Engineer &location=Israel &f_E=2%2C3%2C4 # רמות ניסיון: Entry, Associate, Mid-Senior &f_TPR=r3600 # פורסם בשעה האחרונה &sortBy=DD # מסודר לפי תאריך (חדש ראשון)
```

שלב ב׳ — גלילת הדף וסריקת כרטיסיות

הסוכן גולל את דף תוצאות החיפוש למטה (כדי שLinkedIn יטען עוד משרות) ואז קורא את כל כרטיסיות המשרות שמוצגות.

שלב ג׳ — סינון כפילויות

לכל משרה שנמצאת, הסוכן בודק את ה-job_id שלה. אם המזהה כבר קיים ב-already_seen (שנטען מה-DB), הוא מדלג עליה. כך לא מבזבזים זמן על משרות שכבר עובדו בעבר.

שלב ד׳ — קריאת תיאורים מלאים

כרטיסיית המשרה מכילה רק פרטים בסיסיים. כדי שה-AI יוכל לדרג, צריך את התיאור המלא. הסוכן נכנס לדף כל משרה בנפרד ומחלץ את הטקסט.

> ⏱️ **מגבלת זמן — max_jobs_per_run**
>
> כדי שהסוכן לא ירוץ יותר מדי זמן, יש מגבלה על כמות המשרות שיעבד בכל ריצה (ברירת מחדל: 30). אפשר לשנות ב-config.yaml.

פונקציות עיקריות

פונקציה	מה עושה
`search_jobs()`	הפונקציה הראשית — מחזירה רשימת Job objects חדשים
`get_job_description()`	נכנסת לדף המשרה וחולצת את התיאור המלא
`_scrape_job_card()`	קוראת נתוני בסיס מכרטיסיית משרה
`_build_search_url()`	בונה כתובת URL לחיפוש
`_extract_job_id()`	מחלצת את המזהה המספרי מ-URL

פרק 6

filter.py — מנוע הדירוג (AI)

filter.py — Claude AI Scoring

src/filter.py

זהו "הלב החכם" של הסוכן. עבור כל משרה חדשה, מודול זה שולח בקשה ל-Claude AI עם ה-CV שלך ותיאור המשרה. Claude מחזיר ציון 0-100 והסבר בעברית. רק משרות שמקבלות ציון מעל הסף עוברות לשליחה במייל.

איך Claude מדרג משרות?

הסוכן שולח ל-Claude הנחיה מדויקת (Prompt) שמבקשת ממנו לפעול כ"יועץ קריירה" ולדרג את ההתאמה לפי שני קריטריונים:

📋 קריטריון 1 — התאמת CV

עד כמה הכישורים, הניסיון והרקע של המועמד תואמים לדרישות המשרה הספציפית.

🌍 קריטריון 2 — רלוונטיות לתחום

האם התפקיד שייך לאותו תחום מקצועי? גם אם הכלים לא מוזכרים ב-CV, משרה בתחום קרוב תקבל ציון גבוה.

סולם הציונים

טווח ציון	משמעות	צבע
100–90	התאמה מצוינת — CV תואם ישירות + תחום מדויק	ירוק
89–70	התאמה טובה — רוב הדרישות מתאימות, תחום רלוונטי	כתום
69–50	התאמה חלקית — תחום סמוך, חפיפה חלקית	מתחת לסף ← לא נשלח
49–0	לא מתאים — תחום לא קשור	מתחת לסף ← לא נשלח

📝 ברירת המחדל — ציון מינימלי 70

ניתן לשנות את הסף ב-config.yaml. ככל שהסף גבוה יותר, כך יגיעו אליך פחות משרות אך איכותיות יותר.

הדיאלוג עם Claude (Prompt)

Claude מקבל הנחיה מדויקת ומחזיר תשובה בפורמט JSON בלבד, שמכיל ציון ומשפט הסבר בעברית:

```
// דוגמה לתשובה שClaude מחזיר: { "score": 85, "reason": "המתאים Python Backend תפקיד
שמופיע ב-CV" } ו-Data Engineering לניסיון של 3 שנים בפיתוח API
```

הגנה מפני שליחה כפולה

לפני שסוכן שולח משרה לדירוג, הוא בודק ב-tracker אם המשרה כבר נשלחה בעבר. אם כן — היא מדולגת לגמרי ולא נשלחת שוב.

פונקציות עיקריות

פונקציה	מה עושה
`filter_jobs()`	הפונקציה הראשית — מדרגת את כל המשרות ומחזירה רק הרלוונטיות
`score_job()`	שולחת משרה בודדת לClaude ומחזירה (score, reason)
`load_cv_()`	קוראת את קובץ cv.txt

💰 **עלות API:**

כל משרה שנשלחת ל-Claude עולה כסף (מאוד קטן — כמה סנטים לריצה שלמה). לכן, המודול מעביר רק את 3,000 התווים הראשונים של תיאור המשרה כדי לחסוך.

פרק 7

notifier.py — שולח המיילים

notifier.py — Email Digest

src/notifier.py

מודול זה אחראי על בניית המייל המעוצב ושליחתו. הוא יוצר מייל עם עיצוב HTML יפה (רגיל + גרסת טקסט לגיבוי) ושולח דרך Gmail SMTP.

מבנה המייל שנשלח

המייל כולל טבלה עם כל המשרות הרלוונטיות, כשלכל משרה יש:

שם התפקיד
קישור לדף המשרה

שם החברה
מיקום המשרה

ציון AI (0–100)
ירוק / כתום לפי ציון

הסבר בעברית
למה המשרה רלוונטית

צבעי הציון במייל

85+	מצוין	70–84	טוב	<70 לא נשלח

שיטת השליחה — Gmail SMTP

SMTP הוא הפרוטוקול הסטנדרטי לשליחת מיילים באינטרנט. המודול מתחבר לשרת Gmail (`smtp.gmail.com:465`) עם "App Password" (סיסמה ייעודית לאפליקציות חיצוניות, לא סיסמת Gmail רגילה) ושולח את המייל בצורה מוצפנת (SSL).

לוגיקת "מה שולחים"

מצב	send_if_empty	מה קורה?
נמצאו משרות רלוונטיות	כל מצב	מייל נשלח עם הטבלה
אין משרות רלוונטיות	false (ברירת מחדל)	מייל לא נשלח בכלל
אין משרות רלוונטיות	true	מייל נשלח עם הודעה שלא נמצא כלום
dry-run	כל מצב	מייל לא נשלח בכלל

✅ **ערובה לשליחה חד-פעמית:**

אחרי שמייל נשלח, כל המשרות שנכללו בו מסומנות ב-DB כ"נשלחו". ה-filter.py בודק זאת בריצות הבאות ומדלג עליהן אוטומטית — לעולם לא תקבל אותה משרה פעמיים.

פרק 8

tracker.py — מסד הנתונים

tracker.py — SQLite Database

`src/tracker.py → data/jobs.db`

מודול זה הוא "הזיכרון" של הסוכן — מנהל מסד נתונים SQLite שמאפשר לסוכן לדעת אילו משרות כבר ראה, אילו דרג, ואילו כבר שלח למשתמש. בלעדיו, הסוכן היה שולח את אותן משרות שוב ושוב.

מה זה SQLite?

SQLite הוא מסד נתונים קל-משקל שמאוחסן כקובץ יחיד (`data/jobs.db`). אין צורך בשרת נפרד — הקובץ הוא מסד הנתונים כולו. מתאים בדיוק לפרויקטים שרצים על מחשב אחד ולא צריכים להתמודד עם כמות עצומה של נתונים.

מבנה טבלת ה-jobs

עמודה	סוג	תיאור
`job_id`	TEXT (מפתח ראשי)	מזהה ייחודי של המשרה ב-LinkedIn
`title`	TEXT	שם התפקיד
`company`	TEXT	שם החברה
`location`	TEXT	מיקום
`apply_url`	TEXT	קישור להגשת מועמדות
`score`	INTEGER	ציון שנתן Claude (0 = לא דורג)
`reason`	TEXT	הסבר בעברית מ-Claude
`emailed`	INTEGER (0/1)	האם נשלח במייל? 0=לא, 1=כן
`seen_at`	TEXT	תאריך ושעה של זיהוי המשרה

פונקציות עיקריות

פונקציה	מה עושה
`init_db()`	יוצרת את הטבלה אם לא קיימת (בטוח לקריאה חוזרת)
`mark_seen()`	מוסיפה משרה חדשה ל-DB (INSERT OR IGNORE — לא דורסת)
`is_seen()`	האם המשרה כבר ב-DB?
`mark_emailed()`	מסמנת רשימת job_ids כ-"נשלחו"
`is_emailed()`	האם המשרה כבר נשלחה בעבר?
`get_all_seen_ids()`	מחזירה SET של כל המזהים — לסינון מהיר
`get_recent_jobs()`	מחזירה 50 המשרות האחרונות (לממשק הווב)
`get_stats()`	מחזירה סטטיסטיקות: כמה נראו, כמה נשלחו

🛡️ **הגנה מפני כפילויות — INSERT OR IGNORE**

כשמנסים להוסיף משרה שכבר קיימת (לפי job_id), מסד הנתונים פשוט מתעלם מהבקשה מבלי לגרום לשגיאה. זה מבטיח שאין כפילויות אפילו אם אותה משרה מופיעה בחיפושים שונים.

ההבדל בין "נראה" ל"נשלח"

👁️ **"נראה" (seen)**

משרה שהסוכן גילה בחיפוש. גם משרות שדורגו נמוך מסומנות כנראו — הסוכן לא יעבד אותן שוב.

📤 **"נשלח" (emailed)**

משרה שדורגה גבוה וכלולה במייל שנשלח בפועל. לעולם לא תיכלל במייל עתידי.

פרק 9

app.py — ממשק הניהול (Web UI)

app.py — Flask Web Dashboard

`app.py → http://localhost:5000`

ממשק גרפי מקומי (רץ בדפדפן שלך) שמאפשר לשלוט בסוכן ולראות את כל הנתונים — בלי לפתוח טרמינל ולהקליד פקודות.

מה ניתן לעשות בממשק?

▶ הרץ עכשיו

מפעיל ריצה מיידית עם שליחת מייל אמיתית

Dry Run

ריצת בדיקה — מחפש ומדרג, לא שולח מייל

Scheduler

הפעלה/עצירה של ריצות אוטומטיות

הגדרות

עריכת מילות חיפוש, ציון מינימלי, תדירות

סטטיסטיקות

כמה משרות נראו, נשלחו, ומתי הריצה האחרונה

לוג חי

צפייה ב-120 שורות אחרונות של הלוג בזמן אמת

ארכיטקטורה טכנית — Flask + Threads

ממשק הווב בנוי על Flask — ספריית Python פשוטה לבניית אתרים. הסוכן והממשק רצים **במקביל** באמצעות Threads (תהליכי ריצה מקביליים):

```
Thread 1: Flask web server
→ מגיב לבקשות HTTP
→ מציג את ממשק הווב
→ מקבל פקודות מהמשתמש
```

⇄

```
Thread 2: Agent runner
→ מריץ את הסוכן
→ מעדכן state מרכזי
→ הממשק רואה את הסטטוס
```

הממשק לא "קופא" כשהסוכן רץ — הוא ממשיך לאפשר גלישה וצפייה בסטטוס בזמן שהסוכן עובד ברקע. ה-state המרכזי (סטטוס, תוצאות אחרונות) מוגן עם Lock כדי למנוע בעיות של כתיבה מקבילה.

API Endpoints

נתיב	מתודה	פעולה
/	GET	דף הבית — מציג את כל הממשק
`api/run/`	POST	מפעיל ריצה (dry_run=true/false)
`api/scheduler/start/`	POST	מפעיל את ה-Scheduler
`api/scheduler/stop/`	POST	עוצר את ה-Scheduler
`api/config/`	GET/POST	קריאה/שמירה של הגדרות
`api/state/`	GET	סטטוס נוכחי (בשימוש polling)
`api/logs/`	GET	120 שורות אחרונות של הלוג
`api/stats/`	GET	סטטיסטיקות DB

🔄 **Polling בזמן ריצה:**

כשהסוכן רץ, הממשק שואל את ה-API כל 3 שניות (`api/state/`) לסטטוס עדכני. ברגע שהריצה מסתיימת, הדף מתרענן אוטומטית.

פרק 10

config.yaml ו-.env — קבצי ההגדרות

config.yaml — הגדרות הריצה

קובץ זה שולט בהתנהגות הסוכן. אפשר לערוך אותו ישירות או דרך ממשק הווב.

```
# config.yaml — הגדרות חיפוש search: keywords: - "Software Engineer" # מילות חיפוש
ב-LinkedIn - "Python Developer" location: "Israel" # מיקום לחיפוש experience_levels:
# 2 # "2" - :רמות ניסיון=Entry Level - "3" # 3=Associate - "4" # 4=Mid-Senior
max_jobs_per_run: 30 # מקסימום משרות לריצה schedule_hours: 1 # כל כמה שעות לרוץ
filter: min_relevance_score: 70 # ציון מינימלי לשליחה cv_path: "cv.txt" # נתיב ל-CV
notifications: email_to: "you@gmail.com" # לאיזה מייל לשלוח email_from:
"bot@gmail.com" # מאיזה מייל לשלוח send_if_empty: false # שלח גם אם לא נמצא?
```

רמות ניסיון של LinkedIn

קוד	רמה בעברית	משמעות
1	Internship	סטאז' / תוכניות סטודנטים
2	Entry Level	ללא ניסיון / עד שנתיים
3	Associate	1–3 שנות ניסיון
4	Mid-Senior	3–10 שנות ניסיון
5	Director	תפקיד ניהולי בכיר
6	Executive	VP / C-Level

.env — קובץ הסיסמאות

🔒 **חשוב מאוד — קובץ .env לעולם לא מועלה לאינטרנט!**

הקובץ מכיל סיסמאות ומפתחות API. הוא נמצא ברשימת .gitignore ולא יועלה ל-GitHub.

```
# .env — (פרטי לחלוטין!) פרטי גישה # LinkedIn account
LINKEDIN_EMAIL=your_email@gmail.com LINKEDIN_PASSWORD=your_password # Claude AI API key (מ-console.anthropic.com) ANTHROPIC_API_KEY=sk-ant-... # Gmail for sending emails GMAIL_SENDER=your_gmail@gmail.com GMAIL_APP_PASSWORD=xxxx xxxx xxxx xxxx # לא סיסמת Gmail רגילה!
```

🔑 Gmail App Password — מה זה?

Google לא מאפשר לאפליקציות צד שלישי להשתמש בסיסמה הרגילה שלך. צריך ליצור "App Password" ייעודי ב-Google Account > Security > 2-Step Verification > App passwords.

פרק 11

הגנה מפני זיהוי כבוט

LinkedIn (כמו רוב האתרים הגדולים) מנסה לזהות ולחסום תוכנות אוטומטיות. הסוכן מיישם מספר טכניקות כדי להיראות כגולש אנושי אמיתי:

1. דפדפן גלוי (Non-Headless)

רוב הבוטים משתמשים ב-"headless browser" — דפדפן ללא חלון גרפי. LinkedIn יודע לזהות זאת. הסוכן מריץ דפדפן **עם חלון גרפי גלוי**, בדיוק כמו גלישה רגילה.

2. User Agent אמיתי

כל בקשת HTTP כוללת מחרוזת שמזהה את הדפדפן (User Agent). הסוכן מגדיר User Agent זהה לChrome אמיתי:

```
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/124.0.0.0 Safari/537.36
```

3. השהיות אקראיות

בין כל פעולה ופעולה, הסוכן ממתין זמן **אקראי** (לא קבוע). אדם אמיתי לא קורא/לוחץ בקצב מדוייק — הסוכן מחקה זאת:

בין כרטיסיות:	בין דפים:
0.3 – 0.8 שניות	1.5 – 4 שניות

4. עוגיות שמורות

במקום להתחבר מחדש בכל ריצה (התנהגות חשודה), הסוכן שומר את ה-session cookies ומשתמש בהן בריצות הבאות — בדיוק כמו שדפדפן רגיל עושה.

5. ביטול דגל האוטומציה

Chromium מוסיף ב-default דגל `--enable-automation` שאתרים יכולים לזהות. הסוכן מבטל זאת עם הדגל `--disable-blink-features=AutomationControlled`.

6. טיפול ב-2FA / CAPTCHA

אם LinkedIn מזהה כניסה חשודה ומציג אתגר אבטחה, הסוכן **עוצר 60 שניות** ומאפשר לך להשלים את האימות ידנית (בחלון הדפדפן הגלוי) לפני שממשיך.

פרק 12

🚀 מדריך התקנה והפעלה

דרישות מוקדמות

✅ Python 3.11 ומעלה | ✅ חשבון LinkedIn | ✅ מפתח API של Gmail ✅ | Anthropic (claude.ai/api) עם App Password

שלבי התקנה

📥 שלב 1 — התקנת ספריות

```
pip install -r requirements.txt python -m playwright install chromium
```

📝 שלב 2 — יצירת קובץ .env

העתק את .env.example ל-.env ומלא את הפרטים:

```
LINKEDIN_EMAIL=your@email.com LINKEDIN_PASSWORD=yourpassword
ANTHROPIC_API_KEY=sk-ant-... GMAIL_SENDER=sender@gmail.com
GMAIL_APP_PASSWORD=xxxx xxxx xxxx xxxx
```

📄 שלב 3 — הכנת CV

שמור את ה-CV שלך כ-**cv.txt** בתיקיית הפרויקט. טקסט רגיל, לא Word/PDF.

⚙️ שלב 4 — הגדרת config.yaml

ערוך את מילות החיפוש, המיקום, רמות הניסיון וכתובת המייל לקבלה.

אפשרויות הפעלה

פקודה	מתי להשתמש
`python app.py`	ממשק ווב — הדרך המומלצת. פתח http://localhost:5000
`python main.py --dry-run`	בדיקה ראשונה — לראות שהכל עובד בלי לשלוח מייל
`python main.py --once`	ריצה חד-פעמית עם שליחת מייל
`python main.py`	מצב Scheduler — רץ כל שעה אוטומטית

📌 סיכום — מה הסוכן עושה בשבילך

- 🔍 **מחפש** משרות ב-LinkedIn לפי הקריטריונים שהגדרת
- 🤖 **מדרג** כל משרה עם AI ביחס ל-CV שלך
- 📧 **שולח** מייל עם רק המשרות הכי רלוונטיות
- 💾 **זוכר** מה שלח — לעולם לא שולח אותה משרה פעמיים
- ⏰ **פועל אוטומטית** כל שעה (או לפי הגדרה)
- 🖥️ **נשלט** דרך ממשק ווב נוח ב-localhost:5000